

HIVE

Introduction to Hive

Apache Hive is a data warehouse infrastructure tool built on top of Hadoop for processing and analyzing large amounts of structured data. It provides an easy way to perform data analysis using a SQL-like language called **HiveQL (HQL)**.

Hive simplifies Big Data processing by converting HiveQL queries into Hadoop MapReduce jobs internally. Therefore, users do not need to write complex MapReduce programs in Java.

Hive was initially developed by **Facebook** to manage and analyze huge volumes of data. Later, it became an open-source project under the **Apache Software Foundation**. Today, many organizations such as Amazon use Hive for Big Data analytics.

Need for Hive

Writing MapReduce programs directly in Java is difficult and time-consuming. Hive was introduced to simplify this process.

Hive allows users to:

- query large datasets easily,
- perform data analysis using SQL-like commands,
- and process Big Data without writing complex code.

It is mainly used for:

- data warehousing,
 - batch processing,
 - reporting,
 - and analytics.
-

Features of Hive

The important features of Hive are given below:

1. SQL-Like Query Language

Hive provides a SQL-like language called **HiveQL (HQL)** for querying and analyzing data.

Example:

```
SELECT * FROM employee;
```

This makes Hive easy to learn for users familiar with SQL.

2. Distributed Data Processing

Hive processes huge datasets stored in Hadoop Distributed File System (HDFS).

It converts HiveQL queries into:

- MapReduce jobs,
 - Spark jobs,
 - or Tez jobs.
-

3. Scalable

Hive can handle very large datasets by adding more nodes to the Hadoop cluster.

4. Schema Management

Hive stores:

- table information,
- column names,
- data types,
- and metadata

inside a component called **Metastore**.

The actual data is stored in HDFS.

5. Designed for OLAP

Hive is mainly designed for:

- Online Analytical Processing (OLAP),

- reporting,
- and data analysis.

It is not suitable for transaction processing.

6. Supports Structured Data

Hive mainly works with structured data stored in tabular format.

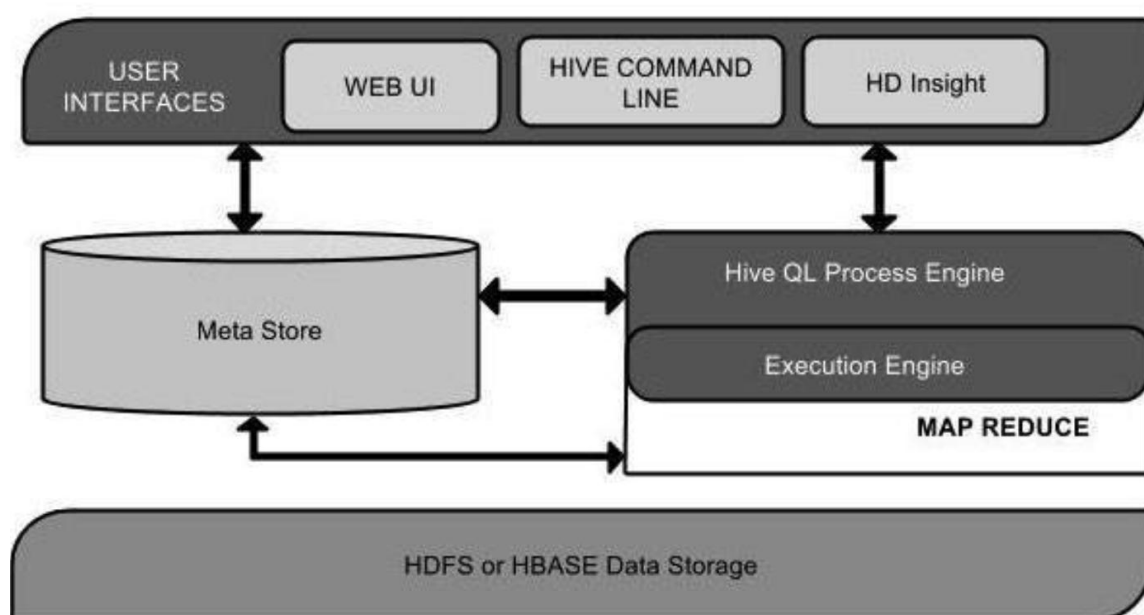
7. Extensible

Users can create:

- User Defined Functions (UDFs),
- custom scripts,
- and plugins.

Architecture of Hive

The following component diagram depicts the architecture of Hive:

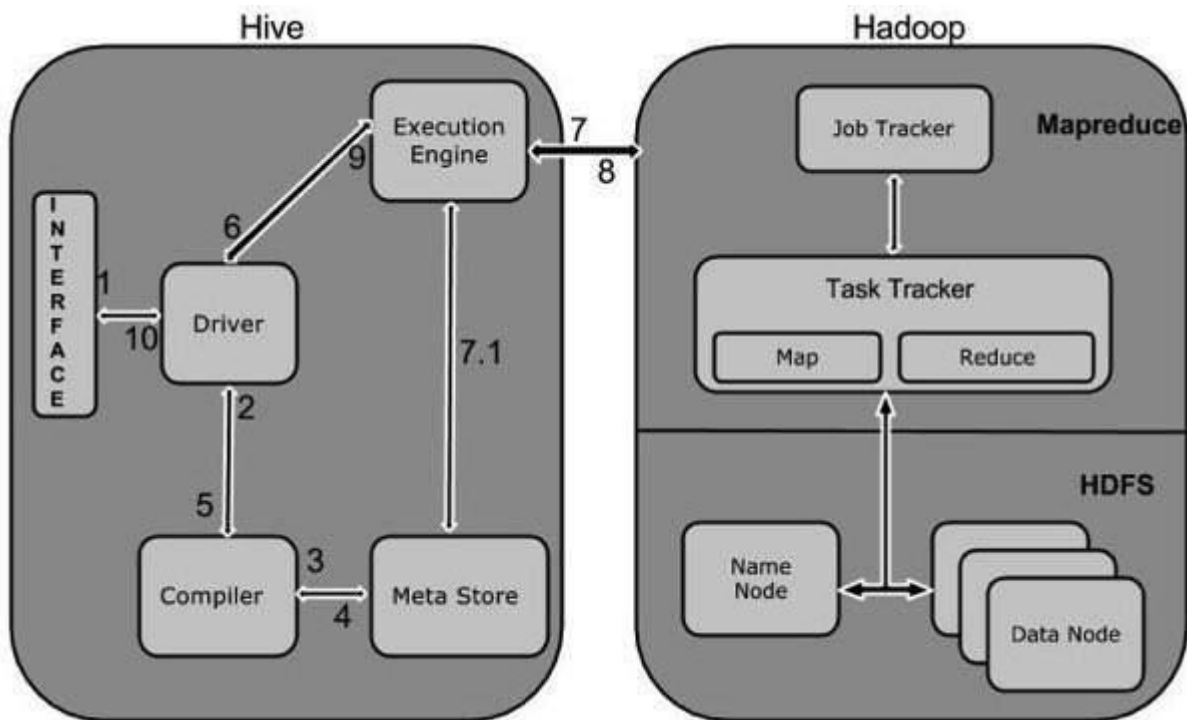


This component diagram contains different units. The following table describes each unit:

Unit Name	Operation
User Interface	Hive is a data warehouse infrastructure software that can create interaction between user and HDFS. The user interfaces that Hive supports are Hive Web UI, Hive command line, and Hive HD Insight (In Windows server).
Meta Store	Hive chooses respective database servers to store the schema or Metadata of tables, databases, columns in a table, their data types, and HDFS mapping.
HiveQL Process Engine	HiveQL is similar to SQL for querying on schema info on the Metastore. It is one of the replacements of traditional approach for MapReduce program. Instead of writing MapReduce program in Java, we can write a query for MapReduce job and process it.
Execution Engine	The conjunction part of HiveQL process Engine and MapReduce is Hive Execution Engine. Execution engine processes the query and generates results as same as MapReduce results. It uses the flavor of MapReduce.
HDFS or HBASE	Hadoop distributed file system or HBASE are the data storage techniques to store data into file system.

Working of Hive

The following diagram depicts the workflow between Hive and Hadoop.



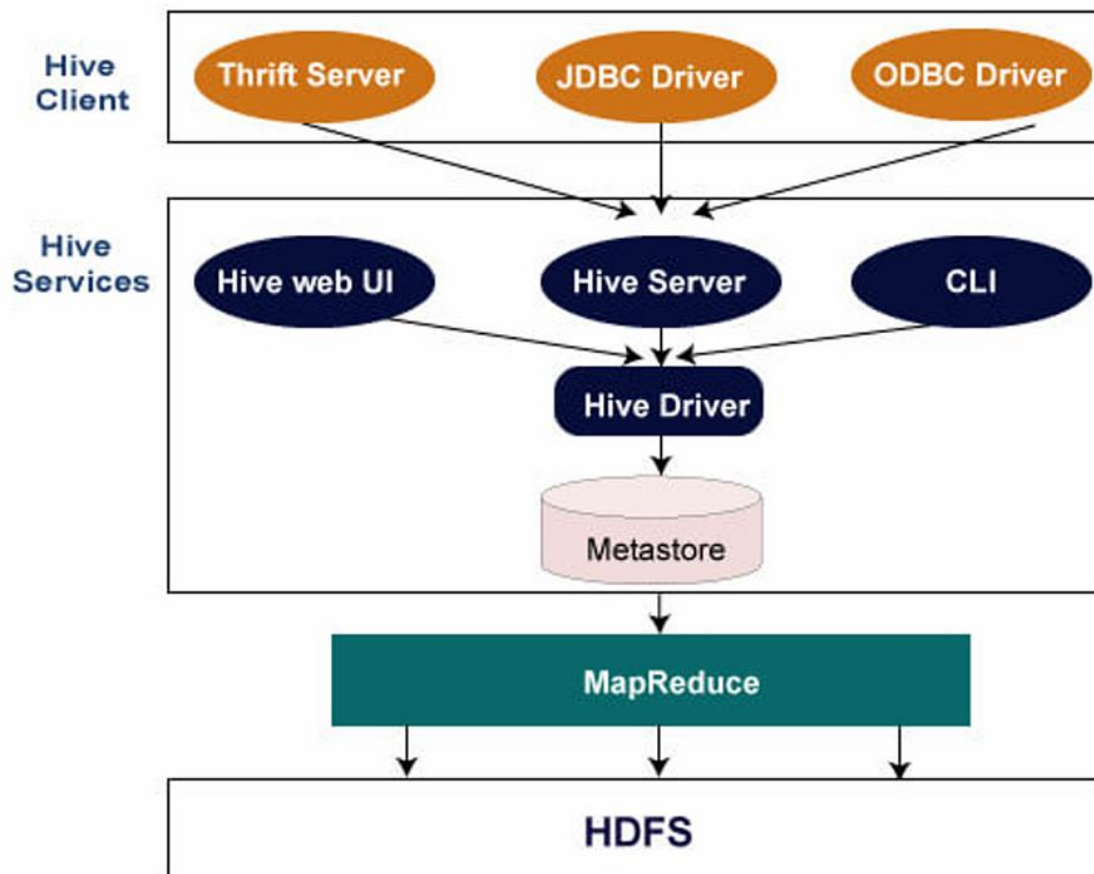
The following table defines how Hive interacts with Hadoop framework:

Step No.	Operation
1	Execute Query The Hive interface such as Command Line or Web UI sends query to Driver (any database driver such as JDBC, ODBC, etc.) to execute.
2	Get Plan The driver takes the help of query compiler that parses the query to check the syntax and query plan or the requirement of query.
3	Get Metadata The compiler sends metadata request to Metastore (any database).
4	Send Metadata

	Metastore sends metadata as a response to the compiler.
5	Send Plan The compiler checks the requirement and resends the plan to the driver. Up to here, the parsing and compiling of a query is complete.
6	Execute Plan The driver sends the execute plan to the execution engine.
7	Execute Job Internally, the process of execution job is a MapReduce job. The execution engine sends the job to JobTracker, which is in Name node and it assigns this job to TaskTracker, which is in Data node. Here, the query executes MapReduce job.
7.1	Metadata Ops Meanwhile in execution, the execution engine can execute metadata operations with Metastore.
8	Fetch Result The execution engine receives the results from Data nodes.
9	Send Results The execution engine sends those resultant values to the driver.
10	Send Results The driver sends the results to Hive Interfaces.

Hive Architecture

The following architecture explains the flow of submission of query into Hive



Hive Client

Hive supports application development in different programming languages such as:

- Java
- Python
- C++

To allow these applications to communicate with Hive, different types of Hive Clients are provided.

Hive Clients act as interfaces between the user/application and the Hive system.

Types of Hive Clients

1. Thrift Server

Thrift Server is a cross-language service platform that allows applications written in different programming languages to connect with Hive.

It accepts requests from applications that support the **Thrift protocol**.

Features

- Supports multiple programming languages
- Enables communication between external applications and Hive
- Provides remote access to Hive services

Applications

- Connecting Python applications to Hive
 - Distributed client-server communication
-

2. JDBC Driver

JDBC (Java Database Connectivity) Driver is used to establish a connection between Hive and Java applications.

The JDBC Driver class used in Hive is:

`org.apache.hadoop.hive.jdbc.HiveDriver`

Features

- Allows Java programs to execute HiveQL queries
- Retrieves query results from Hive
- Supports database connectivity for Java applications

Applications

- Java-based analytics applications
 - Enterprise software integration
-

3. ODBC Driver

ODBC (Open Database Connectivity) Driver allows applications supporting the ODBC protocol to connect with Hive.

Features

- Provides connectivity to external reporting tools
- Supports cross-platform database access
- Enables integration with BI tools

Applications

- Power BI
- Excel
- Tableau

Summary of Hive Clients

Hive Client	Purpose
Thrift Server	Supports multi-language communication
JDBC Driver	Connects Java applications to Hive
ODBC Driver	Connects external analytics and BI tools

Hive Services

Hive provides several internal services for processing HiveQL queries and managing data stored in Hadoop.

These services help Hive interact with:

- users,
 - applications,
 - Hadoop,
 - and HDFS.
-

Main Hive Services

1. Hive CLI (Command Line Interface)

Hive CLI is a shell used to execute Hive queries and commands.

Users can directly type HiveQL commands in the command-line environment.

Example

```
SELECT * FROM employee;
```

Features

- Simple command-line environment
 - Direct query execution
 - Useful for administrators and developers
-

2. Hive Web User Interface

Hive Web UI is a web-based graphical interface for executing Hive queries and commands.

It acts as an alternative to Hive CLI.

Features

- Graphical User Interface (GUI)
 - Easy query execution
 - User-friendly environment
-

3. Hive Metastore

Hive Metastore is a central repository that stores metadata information about Hive tables and databases.

Metadata Includes

- Table names
- Column names
- Data types
- Partition information

- HDFS file locations

It also stores:

- serializers,
- deserializers,
- and warehouse structure information.

Importance

Hive uses Metastore to understand:

- where data is stored,
 - how data is organized,
 - and how queries should be executed.
-

4. Hive Server

Hive Server accepts requests from different clients such as:

- JDBC,
- ODBC,
- Thrift,
- and CLI.

It forwards the requests to the Hive Driver for processing.

Hive Server is also called **Apache Thrift Server**.

Features

- Handles client requests
 - Supports remote connections
 - Provides communication between clients and Hive
-

5. Hive Driver

Hive Driver is the main controller of Hive.

It receives queries from:

- Web UI,
- CLI,
- JDBC/ODBC,
- and Thrift clients.

The Driver transfers the query to the compiler for processing.

Functions of Hive Driver

- Receives HiveQL queries
 - Creates execution plans
 - Manages query execution
 - Returns results to the user
-

6. Hive Compiler

The Hive Compiler parses HiveQL queries and performs semantic analysis.

Functions

- Checks query syntax
- Analyzes query structure
- Converts HiveQL queries into MapReduce jobs

The compiler creates an execution plan for the query.

7. Hive Execution Engine

Hive Execution Engine executes the query plan generated by the compiler.

The optimizer generates a logical execution plan in the form of:

- MapReduce tasks
- HDFS tasks

The execution engine executes tasks according to their dependencies.

Functions

- Executes MapReduce jobs

- Coordinates Hadoop processing
- Retrieves results from HDFS

Difference between RDBMS and Hive

RDBMS:

- RDBMS stands for Relational Database Management System.
- RDBMS is a such type of database management system which is specifically designed for relational databases.
- RDBMS is a subset of DBMS. A relational database refers to a database that stores data in a structured format using rows and columns and that structured form is known as table.
- There are some certain rules defined in RDBMS and that are known as Codd's rule.

Hive:

- Hive is a data warehouse software system that provides data query and analysis.
- Hive gives an interface like SQL to query data stored in various databases and file systems that integrate with Hadoop.
- Hive helps with querying and managing large datasets real fast.
- It is an ETL tool for Hadoop ecosystem.

Difference between RDBMS and Hive:

RDBMS	Hive
It is used to maintain database.	It is used to maintain data warehouse.
It uses SQL (Structured Query Language).	It uses HQL (Hive Query Language).
Schema is fixed in RDBMS.	Schema varies in it.
Normalized data is stored.	Normalized and de-normalized both type of data is stored.
Tables in rdms are sparse.	Table in hive are dense.
It doesn't support partitioning.	It supports automation partition.
No partition method is used.	Sharding method is used for partition.

Hive Metastore

The **Hive Metastore (HMS)** is a central repository used to store metadata information about the data stored in Hive warehouse.

Metadata means “data about data”.

The metastore stores information such as:

- table names,
- database names,
- column names,
- data types,
- partition details,
- HDFS file locations,
- users and permissions.

Hive uses this metadata to understand:

- where the data is stored,
- how the data is organized,
- and how queries should be executed.

When a user submits a HiveQL query, Hive first checks the metadata stored in the Metastore before executing the query.

Functions of Hive Metastore

The Hive Metastore performs the following functions:

1. Stores metadata about Hive tables and databases
2. Stores schema information
3. Maintains partition details
4. Stores HDFS data locations
5. Maintains user roles and permissions
6. Helps Hive optimize query execution

Metadata Stored in Hive Metastore

Metadata Information Description

Database Names	Names of Hive databases
Table Names	Names of Hive tables
Column Names	Attributes of tables
Data Types	Type of each column
Partition Information	Details about table partitions
HDFS Location	Physical location of data files
User Permissions	Access control information

Working of Hive Metastore

1. User submits a HiveQL query.
 2. Hive Driver sends request to Metastore.
 3. Metastore provides metadata information.
 4. Hive Compiler creates execution plan.
 5. Query is executed on Hadoop.
 6. Results are returned to the user.
-

Diagram: Working of Hive Metastore



Types of Hive Metastore

Based on the configuration, Hive Metastore can be classified into:

1. Embedded Metastore
2. Local Metastore

1. Embedded Metastore

In Embedded Metastore mode, the Metastore service runs in the same JVM (Java Virtual Machine) as the Hive service.

It uses an embedded Derby database for storing metadata.

Features

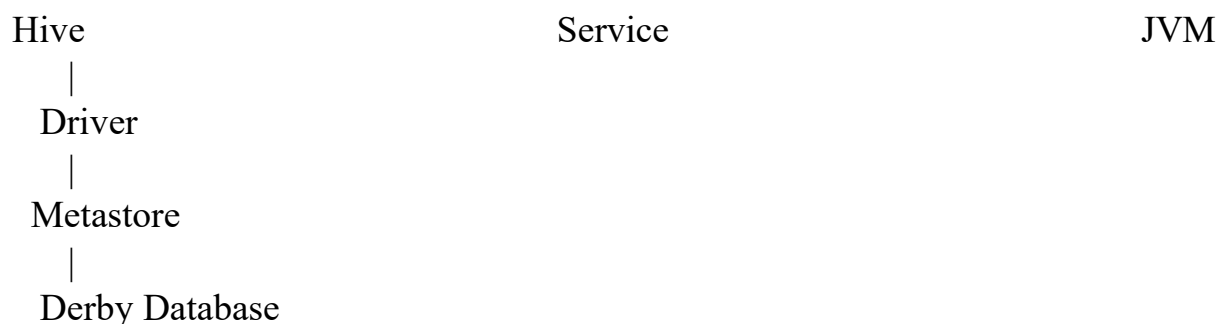
- Simple configuration
- Suitable for testing and small applications
- Hive service and Metastore run together

Limitation

Only one process can connect to the Metastore at a time.

Embedded Metastore Architecture

(Refer to the provided diagram)



2. Local Metastore

In Local Metastore mode, metadata is stored in an external RDBMS such as MySQL.

The Metastore service runs on the same machine as the Hive Server process.

Features

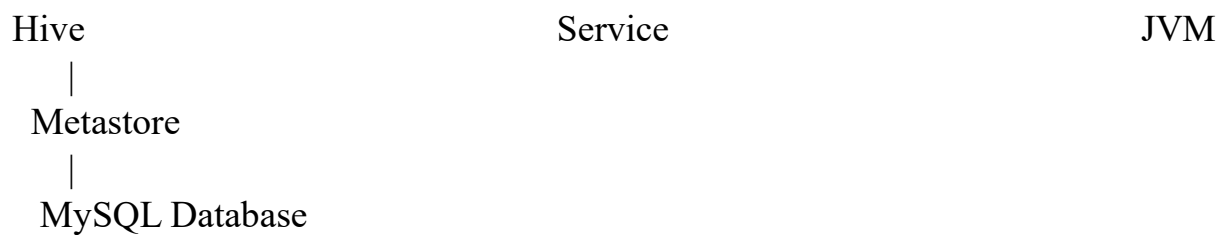
- Supports multiple connections
- Better performance than Embedded mode
- Uses external databases like MySQL

Advantages

- Multi-user support
- Better scalability
- More reliable metadata management

Local Metastore Architecture

(Refer to the provided diagram)



Hive Metastore on AWS

Hive Metastore can also be created on AWS cloud using services such as:

- AWS Glue
- Amazon EMR

1. AWS Glue Data Catalog

AWS Glue provides a fully managed metadata repository called **AWS Glue Data Catalog**.

It can be used as a Hive Metastore.

Features

- Centralized metadata storage

- Managed by AWS
- Supports ETL operations
- Easy integration with Hive

Steps

1. Create AWS Glue ETL job
 2. Configure data source and target
 3. Store metadata in Glue Data Catalog
 4. Query data using HiveQL
-

2. Hive Metastore using Amazon EMR

Amazon EMR (Elastic MapReduce) can also be used to create Hive Metastore.

EMR allows users to:

- launch Hive clusters,
- store data in Amazon S3,
- and manage metadata efficiently.

EMR uses:

- EMRFS (EMR File System)
- Amazon
as the storage backend.

S3

Advantages of Hive Metastore

1. Centralized metadata management
 2. Faster query processing
 3. Easy schema management
 4. Supports partition management
 5. Helps optimize Hive query execution
-

Limitations of Hive Metastore

1. Embedded mode supports only one connection
2. Metadata corruption affects query execution
3. Requires proper database configuration

HiveQL

HiveQL - Select-Where

- The Hive Query Language (HiveQL) is a query language for Hive to process and analyze structured data in a Metastore.
- This chapter explains how to use the SELECT statement with WHERE clause.
- SELECT statement is used to retrieve the data from a table. WHERE clause works similar to a condition.
- It filters the data using the condition and gives you a finite result.
- The built-in operators and functions generate an expression, which fulfils the condition.

Syntax

Given below is the syntax of the SELECT query:

```
SELECT [ALL | DISTINCT] select_expr, select_expr, ...  
FROM table_reference  
[WHERE where_condition]  
[GROUP BY col_list]  
[HAVING having_condition]  
[CLUSTER BY col_list | [DISTRIBUTE BY col_list] [SORT  
BY col_list]]  
[LIMIT number];
```

HiveQL - Select-Order By

- The ORDER BY clause is used to retrieve the details based on one column and sort the result set by ascending or descending order.

Syntax

Given below is the syntax of the ORDER BY clause:

```
SELECT [ALL | DISTINCT] select_expr, select_expr, ...  
FROM table_reference  
[WHERE where_condition]  
[GROUP BY col_list]  
[HAVING having_condition]  
[ORDER BY col_list]  
[LIMIT number];
```

Example

Let us take an example for SELECT...ORDER BY clause. Assume employee table as given below, with the fields named Id, Name, Salary, Designation, and Dept. Generate a query to retrieve the employee details in order by using Department name.

ID	Name	Salary	Designation	Dept
1201	Gopal	45000	Technical manager	TP
1202	Manisha	45000	Proofreader	PR
1203	Masthanvali	40000	Technical writer	TP
1204	Krian	40000	Hr Admin	HR
1205	Kranthi	30000	Op Admin	Admin

The following query retrieves the employee details using the above scenario:

```
hive> SELECT Id, Name, Dept FROM employee ORDER BY  
DEPT;
```

On successful execution of the query, you get to see the following response:

ID	Name	Salary	Designation	Dept
1205	Kranthi	30000	Op Admin	Admin
1204	Krian	40000	Hr Admin	HR
1202	Manisha	45000	Proofreader	PR
1201	Gopal	45000	Technical manager	TP
1203	Masthanvali	40000	Technical writer	TP

```
}
```

Save the program in a file named HiveQLOrderBy.java. Use the following commands to compile and execute this program.

Save the program in a file named HiveQLOrderBy.java. Use the following commands to compile and execute this program.

```
$ javac HiveQLOrderBy.java
```

```
$ java HiveQLOrderBy
```

Output:

ID	Name	Salary	Designation	Dept
1205	Kranthi	30000	Op Admin	Admin
1204	Krian	40000	Hr Admin	HR
1202	Manisha	45000	Proofreader	PR
1201	Gopal	45000	Technical manager	TP
1203	Masthanvali	40000	Technical writer	TP
1204	Krian	40000	Hr Admin	HR

HiveQL - Select-Group By

GROUP BY clause in a SELECT statement. The GROUP BY clause is used to group all the records in a result set using a particular collection column. It is used to query a group of records.

Syntax

The syntax of GROUP BY clause is as follows:

```
SELECT [ALL | DISTINCT] select_expr, select_expr, ...  
FROM table_reference  
[WHERE where_condition]  
[GROUP BY col_list]  
[HAVING having_condition]  
[ORDER BY col_list]  
[LIMIT number];
```

Example

Let us take an example of SELECT...GROUP BY clause. Assume employee table as given below, with Id, Name, Salary, Designation, and Dept fields. Generate a query to retrieve the number of employees in each department.

ID	Name	Salary	Designation	Dept
1201	Gopal	45000	Technical manager	TP

1202	Manisha	45000	Proofreader	PR
1203	Masthanvali	40000	Technical writer	TP
1204	Krian	45000	Proofreader	PR
1205	Kranthi	30000	Op Admin	Admin

The following query retrieves the employee details using the above scenario.

```
hive> SELECT Dept,count(*) FROM employee GROUP BY DEPT;
```

On successful execution of the query, you get to see the following response:

Dept	Count(*)
Admin	1
PR	2
TP	3

HiveQL - Select-Joins

JOIN is a clause that is used for combining specific fields from two tables by using values common to each one. It is used to combine records from two or more tables in the database.

Syntax

```
join_table:
    table_reference JOIN table_factor [join_condition]
  | table_reference {LEFT|RIGHT|FULL} [OUTER] JOIN
table_reference
  join_condition
  | table_reference LEFT SEMI JOIN table_reference
join_condition
  | table_reference CROSS JOIN table_reference
[join_condition]
```

Example

We will use the following two tables in this chapter. Consider the following table named CUSTOMERS..

```
+---+-----+---+-----+-----+
| ID | NAME   | AGE | ADDRESS | SALARY |
+---+-----+---+-----+-----+
| 1  | Ramesh | 32  | Ahmedabad | 2000.00 |
| 2  | Khilan | 25  | Delhi    | 1500.00 |
| 3  | kaushik | 23  | Kota     | 2000.00 |
| 4  | Chaitali | 25  | Mumbai   | 6500.00 |
| 5  | Hardik | 27  | Bhopal   | 8500.00 |
| 6  | Komal  | 22  | MP       | 4500.00 |
| 7  | Muffy  | 24  | Indore   | 10000.00 |
+---+-----+---+-----+-----+
```

Consider another table ORDERS as follows:

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

There are different types of joins given as follows:

- JOIN
- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN

JOIN

JOIN clause is used to combine and retrieve the records from multiple tables. JOIN is same as OUTER JOIN in SQL. A JOIN condition is to be raised using the primary keys and foreign keys of the tables.

The following query executes JOIN on the CUSTOMER and ORDER tables, and retrieves the records:

```
hive> SELECT c.ID, c.NAME, c.AGE, o.AMOUNT  
FROM CUSTOMERS c JOIN ORDERS o  
ON (c.ID = o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

```
+---+-----+---+-----+
| ID | NAME   | AGE | AMOUNT |
+---+-----+---+-----+
| 3 | kaushik | 23 | 3000  |
| 3 | kaushik | 23 | 1500  |
| 2 | Khilan  | 25 | 1560  |
| 4 | Chaitali | 25 | 2060  |
+---+-----+---+-----+
```

LEFT OUTER JOIN

The HiveQL LEFT OUTER JOIN returns all the rows from the left table, even if there are no matches in the right table. This means, if the ON clause matches 0 (zero) records in the right table, the JOIN still returns a row in the result, but with NULL in each column from the right table.

A LEFT JOIN returns all the values from the left table, plus the matched values from the right table, or NULL in case of no matching JOIN predicate.

The following query demonstrates LEFT OUTER JOIN between CUSTOMER and ORDER tables:

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c
LEFT OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

```
+---+-----+-----+-----+
| ID | NAME   | AMOUNT | DATE           |
+---+-----+-----+-----+
| 1 | Ramesh | NULL   | NULL           |
| 2 | Khilan | 1560   | 2009-11-20 00:00:00 |
| 3 | kaushik | 3000   | 2009-10-08 00:00:00 |
| 3 | kaushik | 1500   | 2009-10-08 00:00:00 |
| 4 | Chaitali | 2060   | 2008-05-20 00:00:00 |
| 5 | Hardik | NULL   | NULL           |
| 6 | Komal  | NULL   | NULL           |
| 7 | Muffy  | NULL   | NULL           |
+---+-----+-----+-----+
```

RIGHT OUTER JOIN

The HiveQL RIGHT OUTER JOIN returns all the rows from the right table, even if there are no matches in the left table. If the ON clause matches 0 (zero) records in the left table, the JOIN still returns a row in the result, but with NULL in each column from the left table.

A RIGHT JOIN returns all the values from the right table, plus the matched values from the left table, or NULL in case of no matching join predicate.

The following query demonstrates RIGHT OUTER JOIN between the CUSTOMER and ORDER tables.

```
notranslate"> hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c RIGHT OUTER JOIN ORDERS o ON (c.ID =
o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

```
+-----+-----+-----+
| ID | NAME | AMOUNT | DATE |
+-----+-----+-----+
| 3 | kaushik | 3000 | 2009-10-08 00:00:00 |
| 3 | kaushik | 1500 | 2009-10-08 00:00:00 |
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 |
| 4 | Chaitali | 2060 | 2008-05-20 00:00:00 |
+-----+-----+-----+
```

FULL OUTER JOIN

The HiveQL FULL OUTER JOIN combines the records of both the left and the right outer tables that fulfil the JOIN condition. The joined table contains either all the records from both the tables, or fills in NULL values for missing matches on either side.

The following query demonstrates FULL OUTER JOIN between CUSTOMER and ORDER tables:

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c
FULL OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

```
+-----+-----+-----+
| ID | NAME | AMOUNT | DATE |
+-----+-----+-----+
| 1 | Ramesh | NULL | NULL |
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 |
| 3 | kaushik | 3000 | 2009-10-08 00:00:00 |
| 3 | kaushik | 1500 | 2009-10-08 00:00:00 |
```

```
| 4 | Chaitali | 2060 | 2008-05-20 00:00:00 |
| 5 | Hardik | NULL | NULL |
| 6 | Komal | NULL | NULL |
| 7 | Muffy | NULL | NULL |
| 3 | kaushik | 3000 | 2009-10-08 00:00:00 |
| 3 | kaushik | 1500 | 2009-10-08 00:00:00 |
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 |
| 4 | Chaitali | 2060 | 2008-05-20 00:00:00 |
+-----+-----+-----+
```

Hive - Create Table

Create Table Statement

Create Table is a statement used to create a table in Hive. The syntax and example are as follows:

Syntax

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS]
[db_name.] table_name

[(col_name data_type [COMMENT col_comment], ...)]
[COMMENT table_comment]
[ROW FORMAT row_format]
[STORED AS file_format]
```

Example

Let us assume you need to create a table named **employee** using **CREATE TABLE** statement. The following table lists the fields and their data types in employee table:

Sr.No	Field Name	Data Type
1	Eid	int
2	Name	String
3	Salary	Float
4	Designation	string

The following data is a Comment, Row formatted fields such as Field terminator, Lines terminator, and Stored File type.

```
COMMENT 'Employee details'
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\n'
STORED IN TEXT FILE
```

The following query creates a table named **employee** using the above data.

```
hive> CREATE TABLE IF NOT EXISTS employee ( eid int, name String,
salary String, destination String)
COMMENT 'Employee details'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\n'
STORED AS TEXTFILE;
```

If you add the option IF NOT EXISTS, Hive ignores the statement in case the table already exists.

On successful creation of table, you get to see the following response:

```
OK
Time taken: 5.905 seconds
hive>
```

Alter Table Statement

It is used to alter a table in Hive.

Syntax

The statement takes any of the following syntaxes based on what attributes we wish to modify in a table.

```
ALTER TABLE name RENAME TO new_name
ALTER TABLE name ADD COLUMNS (col_spec[, col_spec ...])
ALTER TABLE name DROP [COLUMN] column_name
ALTER TABLE name CHANGE column_name new_name new_type
ALTER TABLE name REPLACE COLUMNS (col_spec[, col_spec ...])
```

Rename To... Statement

The following query renames the table from **employee** to **emp**.

```
hive> ALTER TABLE employee RENAME TO emp;
```

Drop Table Statement

The syntax is as follows:

```
DROP TABLE [IF EXISTS] table_name;
```

The following query drops a table named **employee**:

```
hive> DROP TABLE IF EXISTS employee;
```

On successful execution of the query, you get to see the following response:

```
OK
Time taken: 5.3 seconds
hive>
```

HIVE- User Defined Functions

User Defined Function to convert to upper case

```
package example.hive.udf;

import org.apache.hadoop.hive.sql.exec.UDF

public final class Ucase extends UDF
{
    public String evaluate(final String word)
    {
        return word.toUpperCase();
    }
}
```

Convert this program to Jar (Uppercase.jar)

Hive> ADD JAR Uppercase.jar

Hive> Create temporary FUNCTION touppercase AS example.hive.udf.Ucase9;

Hive> select touppercase(name) from emp;

Sample queries

(1) CREATE DATABASE - Creates a new database

(2) SHOW DATABASES: Lists all existing databases

```
CREATE DATABASE IF NOT EXISTS bda_lab_2023;
```

```
SHOW DATABASES;
```

```
//OUTPUT
```

```
//      OK
```

```
//      bda_lab_2023
```

```
//      default
```

(3) USE - Sets the current database for subsequent HiveQL statements

```
USE bda_labe_2023;
```

```
//OUTPUT
```

```
//OK
```

(4) CREATE TABLE: Creates a new table with a defined schema

```
CREATE TABLE IF NOT EXISTS student (
```

```
    id INT,
```

```
    name STRING,
```

```
    department STRING,
```

```
    semester INT
```

```
)
```

```
COMMENT 'Students details table'
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE;
```

```
//OUTPUT
```

```
//OK
```

(5) SHOW TABLES: Lists all tables within the current database

```
SHOW TABLES;
```

```
//OUTPUT
```

```
//OK
```

```
//student
```

(6) DESCRIBE: Displays the columns and their data types for a specific table.

```
DESCRIBE student;
```

```
//OUTPUT
```

```
//OK
```

```
//id          int
```

```
//name        string
```

```
//department   string
```

```
//semester     int
```

(7) ALTER TABLE: Modifies the structure of an existing table, such as renaming it or adding columns

```
ALTER TABLE student RENAME TO students;
```

```
SHOW TABLES;
```

```
//OUTPUT
```

```
//OK
```

```
//students
```

```
ALTER TABLE students ADD COLUMNS (reg_num, STRING);
```

```
DESCRIBE students;
```

```
//OUTPUT
```

```
//OK
```

```
//id          int
```

```
//name        string
```

```
//department   string
```

```
//semester     int
```

```
//reg_num      string
```

(8) LOAD DATA: Moves data from a local or HDFS file path into a Hive table.

```
LOAD DATA LOCAL INPATH 'C:/hive/input/student-table.csv' INTO TABLE students;
```

```
//Sample file content
```

```
//1,'AASHISH B KUMAR','AI & DS',6,'NIE23AD001'
```

```
//2,'ABHAY VISWANATH N', 'AI & DS',6,'NIE23AD002'
```

```
//3,'ABHIRAMI BABU', 'AI & DS',6,'NIE23AD003'
```

```
//4,'ADNAN','AI&DS',6,'NIE23AD004'
```

```
//5,'AGNA PAUL',6,'NIE23AD005'
```

```
//OUTPUT
```

```
//OK
```

(9) INSERT: Inserts data into a Hive table, either by appending to existing data (INSERT INTO)

```
INSERT INTO students VALUES ('6','ALKA ASOK', 'AI & DS',6,'NIE23AD006');
```

```
//OUTPUT
```

```
//OK
```

(10) SELECT: Retrieves data from a table.

```
SELECT reg_num, name, department, semester from students where id <5;
```

```
//OUTPUT
```

```
//OK
```

```
//NIE23AD001' 'AASHISH B KUMAR' 'AI & DS' 6
```

```
//NIE23AD002' 'ABHAY VISWANATH N' 'AI & DS' 6
```

```
//NIE23AD003' 'ABHIRAMI BABU' 'AI & DS' 6
```

```
//NIE23AD004' 'ADNAN' 'AI&DS' 6
```

HBase

- HBase is a distributed column-oriented database built on top of the Hadoop file system. It is an open-source project and is horizontally scalable.
- HBase is a data model that is similar to Google's big table designed to provide quick random access to huge amounts of structured data. It leverages the fault tolerance provided by the Hadoop File System (HDFS).
- It is a part of the Hadoop ecosystem that provides random real-time read/write access to data in the Hadoop File System.
- One can store the data in HDFS either directly or through HBase.

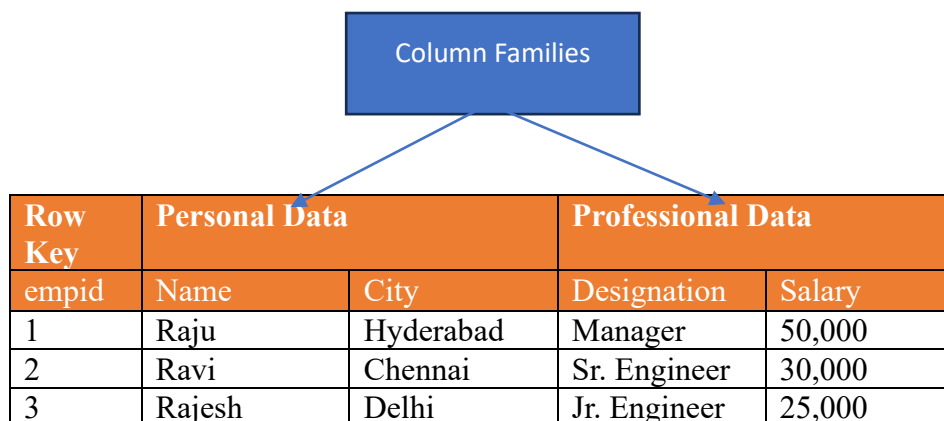
4.3.1 HBasics

Storage Mechanism

- HBase is a column-oriented database.
- The table schema defines only column families.
- A table can have multiple column families and each column family can have any number of columns.
 - Table is a collection of rows.
 - Row is a collection of column families.
 - Column family is a collection of columns.
 - Column is a collection of key value pairs.

Rowid	Column Family			Column Family			Column Family			Column Family		
	Col1	Col2	Col3	Col1	Col2	Col3	Col1	Col2	Col3	Col1	Col2	Col3
1												
2												
3												

Example:



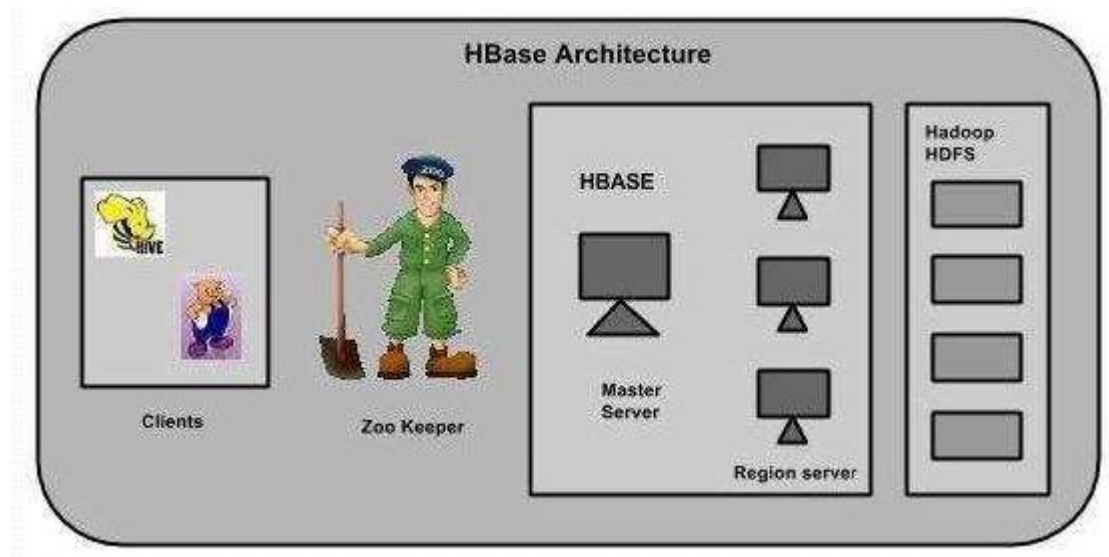
Features

- HBase is linearly scalable.
- It has automatic failure support.
- It provides consistent read and writes.
- It integrates with Hadoop, both as a source and a destination.
- It has easy java API for client.
- It provides data replication across clusters.

Uses

- It is used whenever there is a need to write heavy applications.
- HBase is used whenever we need to provide fast random access to available data.
- Companies such as Facebook, Twitter, Yahoo, and Adobe use Hbase internally.

Architecture



HBase has three major components: the client, a master server, and region servers. Region servers can be added or removed as per requirement.

Master server

- Assigns regions to the region servers and takes the help of Apache ZooKeeper for this task.
- Handles load balancing of the regions across region servers. It unloads the busy region servers and shifts the regions to less occupied servers.
- Maintains the state of the cluster by negotiating the load balancing.
- Is responsible for schema changes and other metadata operations such as creation of tables and column families.

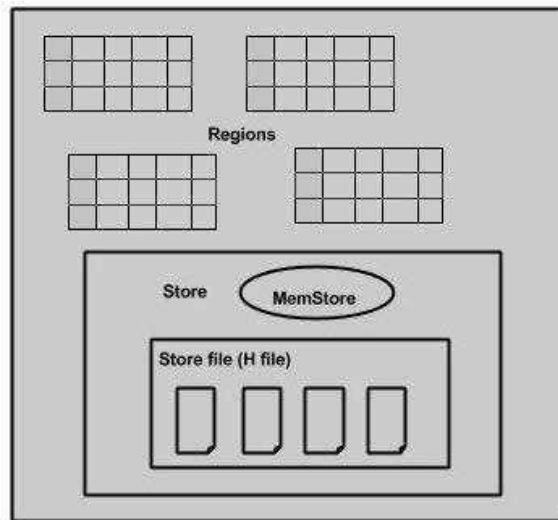
Region server

Regions are nothing but tables. These are split and spread across the region servers.

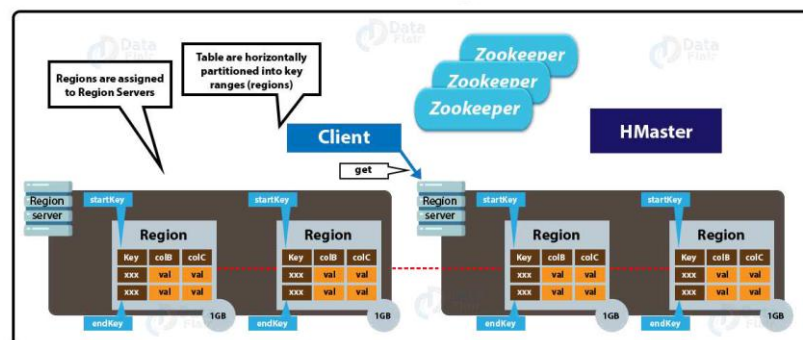
Region server

The region servers looks after

- Data- related operations. Handle read and write requests for all the regions under it.
- Decide the size of the region by following the region size thresholds.



HBase Regions



Zookeeper

- Zookeeper is an open-source project that provides services like maintaining configuration information, naming, distributed synchronization, etc.
- Zookeeper has ephemeral nodes for representing different region servers. Master servers use these nodes to discover region servers.
- Clients communicate with region servers via zookeeper.

Hbase Shell

- HBase contains a shell using which you can communicate with HBase.
- HBase uses the Hadoop File System to store its data.
- It will have a master server and region servers
- The data storage will be in the form of regions
- These regions will be split up and stored in region servers.

DDL Commands

1. create: Creates a table.
2. list: Lists all the tables in HBase.
3. disable: Disables a table.
4. is_disabled: Verifies whether a table is disabled. enable: Enables a table.
5. is_enabled: Verifies whether a table is enabled.
6. describe: Provides the description of a table.
7. alter: Alters a table.
8. exists: Verifies whether a table exists.
9. drop: Drops a table from HBase.

DML Commands

1. put : Put s a cell value at a specified column in a specified row in a part icular table.
2. get : Fetches the contents of row or a cell.
3. delete: Deletes a cell value in a table.
4. deleteall: Deletes all the cells in a given row.
5. scan: Scans and returns the table data.
6. count: Counts and returns the number of rows in a table.
7. truncate: Disables, drops, and recreates a specified table.

Create table

Row key	personal data	professional data

create 'emp', 'personal data', 8professional data9 (correct)

Describe table

describe 'tablename'

Create data

To insert data in an HBase table, the following commands and methods are used:

- put command, (from command line)
- put() method of HTable class, and add() method of Put class. (in program)

Example:

COLUMN FAMILIES				
Row key	personal data		professional data	
empid	name	city	designation	salary
1	raju	hyderabad	manager	50,000
2	ravi	chennai	sr.engineer	30,000
3	rajesh	delhi	jr.engineer	25,000

put 'emp', '1', 'personal data:name', 'Akash'

put 'emp', '1', 'personal data:city', 'mumbai'

put 'emp', '1', 'professional data:designation', 'manager'

put 'emp', '1', 'professional data:salary', '50000'

Display:

scan 'tablename'

Update:

put 'emp', '1', 'personal data:city', 'Delhi'

Read Data:

get 'emp', '1'

get 'emp', '1', {COLUMN=>'personal data :name'}

delete 'emp', '1', 'personal data:city'

Count and truncate

You can count the number of rows of a table using the count command.

count 'emp'

Truncate command disables drops and recreates a table.

truncate 'tablename'

Comparison between Hbase & RDBMS

Sr. No.	Key	RDBMS	HBase
2	SQL	RDBMS uses SQL for data definition & manipulation	HBase is a NO-SQL database
3	Schema	RDBMS has a fixed schema.	HBase has no fixed schema.
4	Orientation	RDBMS is row oriented.	HBase is column oriented.
5	Scalability	RDBMS faces problems in scalability.	HBase is highly scalable.
6	Nature	RDBMS has a more static schema	HBase schema is more dynamic in nature.
7	Data Retrieval	RDBMS data retrieval is slow when the volume becomes high	HBase data retrieval is fast with peta..zetabytes of data.
8	RULE	RDBMS follows ACID(Atomicity, Consistency, Isolation and Durability) Rule.	HBase follows CAP(Consistency, Availability, Partition-tolerance) Rule.
9	Data structure	RDBMS handles structural data.	HBase handles structural, non-structural and semi-structural data.
10	Sparse Data Handling	Sparse data handling is not present.	Sparse data handling is present.

Client API

- Java client API for HBase is used to perform **CRUD** operations on HBase tables.
- HBase is written in Java and has a Java Native API. Therefore JAVA API is used for performing Data manipulation operations.
- **Some of the important classes used for accessing Hbase are:**
 1. Configuration:
 - For configuration details. This class belongs to the **org.apache.hadoop.hbase** package.
 2. Class HTable:
 - HTable is class that represents an HBase table. It is used to create an object to access an HBase table.
 3. Put :

This class is used to perform Put operations for a single row. It belongs to the **org.apache.hadoop.hbase.client** package. This class has constructors and methods like add() for inserting data/updating data.
 4. Get :

This class is used to perform Put operations for a single row. It belongs to the **org.apache.hadoop.hbase.client** package. This class has constructors and method for getting the data from column in a column family
 5. Delete:

This class is used to perform Delete operations on a single row. To delete an entire row, instantiate a Delete object with the row to delete. This class belongs to **org.apache.hadoop.hbase.client** package.
 6. Result:

This class is used to get a single row result of a Get or a Scan query

Client API

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.hbase.*;

public class DataInsert{

    public static void main(String[] args) throws IOException {

        Configuration c = HBaseConfiguration.create(); // Instantiate
        HTable hTable = new HTable(c, "student"); // Instantiate HTable class

        Put P1 = new Put(Bytes.toBytes("1")); // Instantiate put Class

        P1.add(Bytes.toBytes("college"), Bytes.toBytes("id"),Bytes.toBytes("20"));

        P1.add(Bytes.toBytes("college"),Bytes.toBytes("name"),Bytes.toBytes("rishi"
        ));

        hTable.put(P1);

        System.out.println("Data is inserted");

        hTable.close(); // close HTable

    }

}
```

Spark SQL

Introduction to Spark SQL

Apache Spark is a fast and powerful distributed computing framework used for Big Data processing. Spark contains several modules such as:

- Spark Core
- Spark Streaming
- MLlib
- GraphX
- Spark SQL

Among these, **Spark SQL** is a module used for processing structured and semi-structured data.

Spark SQL allows users to perform data analysis using:

- SQL queries,
- DataFrames,
- and Datasets.

It combines the advantages of traditional SQL processing with the distributed computing capability of Apache Spark.

Definition

Spark SQL is a component of Apache Spark that provides support for structured data processing using SQL queries and DataFrame APIs.

It allows users to query large datasets stored in distributed environments efficiently.

Need for Spark SQL

Traditional relational databases cannot efficiently process massive distributed datasets.

Earlier, Hadoop MapReduce was used for Big Data processing, but writing MapReduce programs was:

- complex,
- time-consuming,
- and slower for iterative computations.

Spark SQL was introduced to:

- simplify Big Data querying,
 - provide SQL-like operations,
 - improve processing speed,
 - and support advanced analytics.
-

Features of Spark SQL

1. Supports SQL Queries

Spark SQL allows users to execute SQL queries directly on large datasets.

Example:

```
SELECT * FROM employee;
```

This makes Spark easy for users familiar with SQL.

2. In-Memory Processing

Spark SQL processes data in memory instead of storing intermediate results on disk. This significantly improves performance compared to Hadoop MapReduce.

3. Distributed Processing

Data is distributed across multiple nodes in a cluster and processed in parallel. This enables Spark SQL to handle massive datasets efficiently.

4. Integration with Hive

Spark SQL can integrate directly with:

- Hive tables,
- HiveQL,
- and Hive Metastore.

Users can execute SQL queries on Hive data using Spark.

5. Supports Multiple Data Sources

Spark SQL supports different data formats such as:

- JSON
 - CSV
 - Parquet
 - ORC
 - Hive tables
 - JDBC databases
-

6. Optimized Query Execution

Spark SQL uses a query optimizer called **Catalyst Optimizer** to improve query performance. The optimizer:

- analyzes query plans,
 - reduces unnecessary operations,
 - and improves execution efficiency.
-

7. Scalability

Spark SQL can scale horizontally by adding more nodes to the cluster.

8. Compatibility with Big Data Ecosystem

Spark SQL works with:

- Hadoop HDFS,
- Hive,
- HBase,
- Amazon S3,
- and cloud platforms.

Architecture of Spark SQL

Spark SQL architecture contains several layers responsible for query processing and execution.

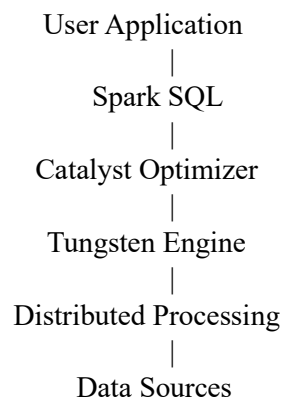
Components of Spark SQL Architecture

Component	Function
Spark Application	User application that submits queries
Spark SQL Engine	Processes structured data
Catalyst Optimizer	Optimizes query execution
Tungsten Engine	Improves memory and CPU efficiency
Data Sources	Provides input data

Working of Spark SQL

1. User submits SQL query or DataFrame operation.
 2. Spark SQL parses the query.
 3. Catalyst Optimizer optimizes the query plan.
 4. Tungsten engine improves execution efficiency.
 5. Query is executed in distributed manner.
 6. Results are returned to the user.
-

Diagram: Spark SQL Architecture



Advantages of Spark SQL

1. Faster than Hadoop MapReduce
2. Easy SQL integration
3. Supports distributed analytics
4. Supports structured and semi-structured data
5. Suitable for machine learning and analytics

Limitations of Spark SQL

1. Requires large memory for in-memory processing
2. Complex cluster setup
3. Performance depends on cluster resources

Applications of Spark SQL

- Data analytics
- Business intelligence
- Machine learning
- Data warehousing
- Real-time analytics

DataFrame API

The DataFrame API is one of the most important features of Spark SQL.

A DataFrame is a distributed collection of data organized into rows and columns similar to a relational database table.

It provides a higher-level abstraction for processing structured data efficiently.

Definition of DataFrame

A DataFrame is a distributed data structure in Spark that stores data in tabular form with named columns.

Characteristics of DataFrame

1. Distributed collection of data
 2. Organized into rows and columns
 3. Immutable in nature
 4. Supports SQL operations
 5. Optimized execution
-

Advantages of DataFrame API

1. High Performance

DataFrames use Catalyst Optimizer and Tungsten engine for faster execution.

2. Easy Syntax

Provides simple APIs for data manipulation.

3. Memory Optimization

Spark optimizes memory usage automatically.

4. Supports Large Datasets

Can process huge distributed datasets efficiently.

5. Integration with SQL

DataFrames can be queried using SQL syntax.

Creating DataFrame

DataFrames can be created from:

- CSV files
 - JSON files
 - Hive tables
 - RDDs
 - Databases
-

Example: Creating DataFrame using PySpark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("DataFrameExample") \
    .getOrCreate()

data = [("Anu",21),("Rahul",22)]

df = spark.createDataFrame(data, ["Name","Age"])

df.show()
```

Output

```
+-----+---+
| Name |Age|
+-----+---+
| Anu  |21 |
```

Common DataFrame Operations

1. show()

Displays DataFrame contents.
df.show()

2. select()

Selects specific columns.
df.select("Name").show()

3. filter()

Filters rows based on conditions.
df.filter(df.Age > 21).show()

4. groupBy()

Groups records based on columns.
df.groupBy("Age").count().show()

5. orderBy()

Sorts records.
df.orderBy("Age").show()

6. withColumn()

Adds or modifies columns.
from pyspark.sql.functions import col

df.withColumn("Age2", col("Age")+1).show()

Spark SQL vs Hive

Spark SQL	Hive
Faster processing	Slower processing
In-memory computation	Disk-based MapReduce
Suitable for real-time analytics	Mainly batch processing
Uses DataFrames	Uses HiveQL
Better iterative processing	Slower iterative processing

SQL Queries on Structured Data using Spark SQL

Spark SQL allows users to execute SQL queries on structured data stored in DataFrames, Hive tables, JSON files, CSV files, and relational databases.

Structured data refers to data organized in rows and columns with a predefined schema.

Spark SQL combines:

- the simplicity of SQL and
- the power of distributed processing in Apache Spark.

Structured Data in Spark SQL

Structured data contains:

- rows,
- columns,
- and predefined data types.

Examples

- Employee records
- Student databases
- Sales reports

Example Table:

ID	Name	Department	Salary
101	Anu	CSE	50000
102	Rahul	ECE	45000

Spark SQL processes such structured data using:

- SQL queries
- and DataFrame API.

Need for SQL Queries in Spark SQL

SQL queries are used because they:

- are easy to understand,
- reduce programming complexity,
- support fast data analysis,
- and allow users familiar with SQL to work with Big Data easily.

Spark SQL internally converts SQL queries into optimized distributed execution plans.

Steps for Executing SQL Queries on Structured Data

1. Create Spark Session
2. Load structured data into DataFrame
3. Create temporary SQL view
4. Execute SQL query
5. Display results

1. Creating Spark Session

Spark Session is the entry point for Spark SQL applications.

Example

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("SparkSQL") \
    .getOrCreate()
```

2. Loading Structured Data

Structured data can be loaded from:

- CSV files
 - JSON files
 - Hive tables
 - Databases
-

Example: Loading CSV File

```
df = spark.read.csv("employee.csv",
                    header=True,
                    inferSchema=True)
```

3. Creating Temporary View

To execute SQL queries, the DataFrame is registered as a temporary SQL table.

```
df.createOrReplaceTempView("employee")
```

Now the DataFrame behaves like a SQL table.

4. Executing SQL Queries

SQL queries are executed using the `spark.sql()` function.

Example Queries on Structured Data

1. Display All Records

```
result = spark.sql("SELECT * FROM employee")
result.show()
```

Output

ID	Name	Department	Salary
101	Anu	CSE	50000
102	Rahul	ECE	45000

2. Select Specific Columns

```
spark.sql("""
SELECT Name, Salary
FROM employee
""").show()
```

3. Filtering Data using WHERE Clause

```
spark.sql("""
SELECT *
FROM employee
WHERE Salary > 45000
""").show()
```

4. Sorting Data using ORDER BY

```
spark.sql("""
SELECT *
FROM employee
ORDER BY Salary DESC
""").show()
```

5. Grouping Data using GROUP BY

```
spark.sql("""
SELECT Department, COUNT(*) AS Total
FROM employee
GROUP BY Department
""").show()
```

6. Aggregate Functions

Spark SQL supports aggregate functions such as:

- COUNT()
- SUM()
- AVG()
- MAX()

- MIN()

Example

```
spark.sql("""  
SELECT AVG(Salary)  
FROM employee  
""").show()
```

7. Using DISTINCT

```
spark.sql("""  
SELECT DISTINCT Department  
FROM employee  
""").show()
```

SQL Queries on JSON Data

Spark SQL can also query JSON structured data.

Example

```
json_df = spark.read.json("employee.json")  
  
json_df.createOrReplaceTempView("emp")  
  
spark.sql("""  
SELECT Name, Salary  
FROM emp  
""").show()
```

SQL Queries on Hive Tables

Spark SQL supports direct querying of Hive tables.

Example

```
spark.sql("""  
SELECT *  
FROM hive_employee  
""").show()
```

Advantages of SQL Queries in Spark SQL

1. Easy query syntax
2. Faster query execution
3. Supports distributed analytics
4. Optimized query processing
5. Supports multiple data sources

Spark SQL Query Optimization

Spark SQL improves performance using:

- Catalyst Optimizer
- Tungsten Execution Engine

These components:

- optimize execution plans,
 - reduce memory usage,
 - and improve processing speed.
-

Applications of SQL Queries in Spark SQL

- Business analytics
 - Data warehousing
 - Report generation
 - Real-time analytics
 - Machine learning preprocessing
-

Spark SQL vs Traditional SQL Databases

Traditional SQL	Spark SQL
Limited scalability	Highly scalable
Single-server processing	Distributed processing
Suitable for small datasets	Suitable for Big Data
Slower for huge datasets	Faster parallel processing

Integration with Hive Metastore

Spark SQL can integrate directly with the Hive Metastore. This allows Spark applications to access:

- Hive databases,
 - Hive tables,
 - partitions,
 - and metadata
- stored in the Hive warehouse.

Using Hive Metastore integration, Spark SQL can execute SQL queries on Hive tables without transferring the data to another system.

What is Hive Metastore?

Hive Metastore is a central metadata repository used by Hive to store information about:

- databases,
- tables,

- columns,
- partitions,
- and HDFS file locations.

Spark SQL uses this metadata to understand:

- table structure,
- schema,
- and data location.

Need for Hive Metastore Integration

Without Hive integration, Spark SQL can only process data loaded manually into DataFrames.

By integrating with Hive Metastore, Spark SQL can:

- directly access Hive tables,
- execute SQL queries on Hive warehouse data,
- reuse existing Hive schemas,
- and support centralized metadata management.

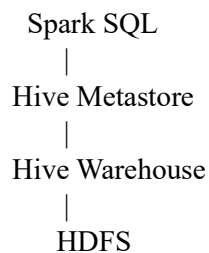
Features of Hive Metastore Integration

1. Access to Hive tables directly
2. Reuse existing Hive metadata
3. Supports HiveQL queries
4. No need to redefine schemas
5. Integration with Hadoop ecosystem
6. Supports distributed query processing

Working of Hive Metastore Integration

1. Spark SQL connects to Hive Metastore.
2. Metadata about Hive tables is retrieved.
3. Spark reads data from HDFS or Hive warehouse.
4. SQL queries are executed using Spark engine.
5. Results are returned to the user.

Architecture of Integration with Hive Metastore



Components Involved

Component	Function
Spark SQL	Executes SQL queries
Hive Metastore	Stores metadata
Hive Warehouse	Stores Hive table data

Component	Function
HDFS	Distributed storage system

Enabling Hive Support in Spark SQL

To integrate Spark SQL with Hive, Hive support must be enabled while creating Spark Session.

Example

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("HiveIntegration") \
    .enableHiveSupport() \
    .getOrCreate()
```

The enableHiveSupport() function allows Spark SQL to connect with Hive Metastore.

Accessing Hive Databases

Spark SQL can display available Hive databases.

Example

```
spark.sql("SHOW DATABASES").show()
```

Using Hive Database

```
spark.sql("USE company")
```

This selects the Hive database named company.

Displaying Hive Tables

```
spark.sql("SHOW TABLES").show()
```

This displays all Hive tables in the selected database.

Querying Hive Tables

Spark SQL can execute SQL queries directly on Hive tables.

Example

```
spark.sql("""
SELECT *
FROM employee
""").show()
```

Creating Hive Table using Spark SQL

Spark SQL can also create Hive-managed tables.

Example

```
spark.sql("""
CREATE TABLE employee(
  id INT,
  name STRING,
  salary INT
)
USING hive
""")
```

Loading Data into Hive Table

```
spark.sql("""
LOAD DATA INPATH '/data/emp.txt'
INTO TABLE employee
""")
```

Advantages of Hive Metastore Integration

1. Direct access to Hive tables
 2. Centralized metadata management
 3. Faster query execution using Spark
 4. Reuse of existing Hive infrastructure
 5. Easy integration with Hadoop ecosystem
 6. Supports large-scale analytics
-

Spark SQL vs Hive Query Execution

Hive	Spark SQL with Hive Metastore
Uses MapReduce	Uses Spark engine
Slower execution	Faster execution
Disk-based processing	In-memory processing
Batch-oriented	Faster analytics

Applications of Hive Metastore Integration

- Big Data analytics
 - Data warehousing
 - Business intelligence
 - Distributed SQL processing
 - Enterprise analytics systems
-

Limitations

- ☐ Requires proper Hive configuration
- ☐ Metadata synchronization issues may occur
- ☐ Large metadata may affect performance

Window Functions in Spark SQL

Window Functions are special SQL functions used to perform calculations across a set of rows related to the current row without grouping the entire table.

Unlike GROUP BY, Window Functions do not combine rows into a single output row. Instead, they perform calculations while still retaining individual rows.

Window functions are widely used in:

- ranking,
- cumulative calculations,
- moving averages,
- and analytical queries.

Definition

A Window Function performs calculations across a group of rows related to the current row while preserving individual row details.

Need for Window Functions

Traditional aggregate functions such as:

- SUM()
- AVG()
- COUNT()

combine multiple rows into a single result.

However, in many analytical applications, we need:

- aggregate calculations,
 - while still keeping each row visible.
- Window Functions solve this problem.

Features of Window Functions

1. Perform calculations across related rows
2. Preserve individual rows
3. Support ranking operations
4. Support cumulative calculations
5. Useful for analytical queries

Window Concept

A **window** is a set of rows related to the current row.

The window is created using the OVER() clause.

Syntax of Window Function

```
function_name() OVER (  
    PARTITION BY column_name  
    ORDER BY column_name  
)
```

Components of Window Function

Component	Purpose
OVER()	Defines the window
PARTITION BY	Divides rows into groups
ORDER BY	Specifies ordering within window
Window Function	Performs calculation

Types of Window Functions

Window Functions are mainly classified into:

1. Ranking Functions
 2. Aggregate Window Functions
 3. Analytical Functions
-

1. Ranking Functions

Ranking functions assign rank values to rows.

Common ranking functions are:

- RANK()
 - DENSE_RANK()
 - ROW_NUMBER()
-

a) ROW_NUMBER()

Assigns a unique sequential number to each row.

Example

Name	Salary	Row_Number
Anu	60000	1
Rahul	50000	2

Query

```
SELECT Name, Salary,
ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RowNum
FROM employee;
```

b) RANK()

Assigns rank to rows. Duplicate values receive the same rank.

Example

Salary	Rank
60000	1
50000	2
50000	2
40000	4

Query

```
SELECT Name, Salary,
RANK() OVER(ORDER BY Salary DESC) AS Rank
FROM employee;
```

c) DENSE_RANK()

Similar to RANK(), but does not skip rank values.

Example

Salary	Dense_Rank
60000	1
50000	2
50000	2
40000	3

Query

```
SELECT Name, Salary,
DENSE_RANK() OVER(ORDER BY Salary DESC) AS DenseRank
FROM employee;
```

2. Aggregate Window Functions

These functions perform aggregate calculations over a window.

Common aggregate window functions:

- SUM()
 - AVG()
 - COUNT()
 - MAX()
 - MIN()
-

Example: SUM()

```
SELECT Name, Salary,  
SUM(Salary) OVER() AS TotalSalary  
FROM employee;  
This calculates total salary while displaying all rows.
```

Example: AVG()

```
SELECT Name, Salary,  
AVG(Salary) OVER() AS AvgSalary  
FROM employee;
```

3. Analytical Functions

Analytical functions compare rows within a window.

Examples:

- LEAD()
 - LAG()
-

a) LEAD()

Returns the next row value.

Query

```
SELECT Name, Salary,  
LEAD(Salary) OVER(ORDER BY Salary) AS NextSalary  
FROM employee;
```

b) LAG()

Returns the previous row value.

Query

```
SELECT Name, Salary,  
LAG(Salary) OVER(ORDER BY Salary) AS PreviousSalary  
FROM employee;
```

PARTITION BY Clause

PARTITION BY divides rows into groups before applying the window function.

Example

```
SELECT Department, Name, Salary,  
RANK() OVER(  
PARTITION BY Department  
ORDER BY Salary DESC  
) AS Rank  
FROM employee;
```

This calculates rank separately for each department.

ORDER BY Clause in Window Functions

ORDER BY specifies the order of rows within the window.

It is mainly used for:

- ranking,
 - cumulative calculations,
 - and analytical functions.
-

Difference Between GROUP BY and Window Functions	
GROUP BY	Window Functions
Combines rows	Preserves rows
Returns grouped result	Returns row-wise result
Used for aggregation	Used for analytics
Less flexible	More flexible

Advantages of Window Functions

1. Supports advanced analytics
 2. Preserves row details
 3. Simplifies complex queries
 4. Useful for ranking and cumulative calculations
 5. Efficient analytical processing
-

Applications of Window Functions

- Ranking students by marks
 - Sales analysis
 - Financial analytics
 - Employee salary comparison
 - Time-series analysis
-

Limitations of Window Functions

1. Complex syntax for beginners
 2. Large windows may affect performance
 3. Requires sorting operations
-